

SkillSync

Workforce Intelligence Platform

Dual-Engine: Discord Observer Bot + Work Engine

Project Report (June 2026)

Comprehensive architecture, implementation details, and system overview

Table of Contents

1. Project Overview
2. System Architecture
3. Database Schema
4. Flask Application
5. API Endpoints
6. Discord Bot
7. ML & Analytics Layer
8. Work Engine
9. Dual Dashboard System
10. Deployment & Infrastructure

1. Project Overview

SkillSync is a workforce intelligence platform that monitors, analyzes, and scores participant behavior across Discord communities and external work management systems. It combines a real-time Discord observer bot with a work engine connector, providing a unified dashboard for community managers and organizational administrators.

The platform operates on two independent authentication systems: Discord OAuth2 for the community-facing dashboard and email/password authentication for the workspace dashboard. A WorkerIdentity model bridges the two domains, enabling behavioral priors from Discord activity to seed workspace scoring models.

Key Features

- Real-time Discord message tracking, moderation event capture, voice activity monitoring
- Behavioral anomaly detection (rule-based + ML via Isolation Forest)
- Burnout risk assessment for staff members
- Federated learning (FedAvg) across Discord guilds
- Work engine connectors for Jira and external project management
- Admin correction feedback loop for score calibration
- Growth analytics: member join/leave tracking with hourly aggregation
- Dual dashboard: community (Discord OAuth2) and workspace (email/password)
- WorkerIdentity bridge linking Discord IDs, Jira Account IDs, Organization Employee IDs

2. System Architecture

Component Overview

The system consists of four main components: a Flask web application serving both dashboards and APIs, a Discord bot for real-time event capture, a work engine for external task integration, and a machine learning layer for predictive analytics.

Flask App (app.py)

Central HTTP server. Hosts community + workspace dashboards, REST API endpoints, serves templates. Auto-applies DB migrations on startup.

Discord Bot (bot.py)

82-line entry point. Core logic in bot_core/ package. Captures messages, voice activity, moderation events, presence changes. Pushes data to Flask API.

Work Engine (work_engine/)

Connectors for Jira and external systems. Syncs tasks, manages work logs, provides external task data to scoring.

ML Layer (ml/)

Anomaly detection, burnout prediction, federated learning, growth forecasting, admin correction feedback. Trains models on activity data.

Data Flow

1. Discord Bot captures events (messages, joins, leaves, voice, moderation) via discord.py library. 2. Bot sends data to Flask API endpoints (/api/observer/*) as JSON payloads. 3. Flask stores data in PostgreSQL via SQLAlchemy models. 4. ML models read from DB, train on scheduled intervals (every 4-6h) or on-demand via API. 5. Dashboards query DB for aggregated stats and render via Jinja2 templates + Chart.js. 6. Work engine polls external systems (Jira) and syncs tasks via API. 7. WorkerIdentity connects Discord profiles to organizational records for cross-domain scoring.

3. Database Schema

PostgreSQL database with 22 tables managed via Flask-SQLAlchemy and Flask-Migrate (Alembic).

Worker

Core participant record. Fields: discord_id, name, score (float), guild_id, last_active, last_seen, created_at.

Task

Work item tracking. Fields: worker_id (FK), title, description, status, source, external_id, created_at, completed_at.

ScoreLog

Audit trail for score changes. Fields: worker_id (FK), change (float), reason, source, created_at.

MessageRecord

Individual Discord messages. Fields: discord_id, name, guild_id, channel_id, message_length, hour_of_day, day_of_week, created_at.

VoiceActivity

Voice channel sessions. Fields: discord_id, guild_id, channel_id, duration_seconds, created_at.

BehavioralAnomaly

Flagged behavioral issues. Fields: discord_id, name, guild_id, anomaly_type, severity, details, source (discord|work_engine), detected_at, cleared_at.

BurnoutRisk

Staff burnout assessment. Fields: worker_id (FK), discord_id, name, score, signals, detected_at.

AdminCorrection

Admin score corrections. Fields: worker_id (FK), admin_id, old_score, new_score, reason, created_at.

Organisation

Workspace tenant. Fields: name, slug (unique), api_key, plan, privacy toggles, created_at.

OrgMember

Workspace user. Fields: org_id (FK), email, name, role (admin|hr|member), password_hash, last_login.

WorkerIdentity

Cross-domain identity bridge. Fields: worker_id (FK, nullable), org_id (FK), discord_id, org_employee_id, jira_account_id, consent flags.

Additional tables: GuildInfo, GuildMember, GuildChannel, GuildRole, AutoModRule, AutoModTrigger, PendingBan, PendingTimeout, MemberJoinLeave, CommunityEvent, PingJoinEvent, RoleChangeLog, MentionRecord.

4. Flask Application

Flask application auto-applies migrations on startup from migrations/ directory. Configured via Config class and .env file.

Blueprints

Blueprint	Prefix	Endpoints
dashboard_bp	/	Community dashboard, guild pages, worker pages
auth_bp	/auth	Discord OAuth2 login/logout
api_bp	/api	Community API endpoints
community_bp	/api	Community-specific API
observer_bp	/api	Bot observer endpoints (activity, moderation, ML)
work_bp	/api	Work engine API (tasks, connectors)
workspace_bp	/workspace	Workspace auth + dashboard (9 routes)

Workspace Routes (routes/workspace.py)

Route	Method	Description
/workspace/login	GET/POST	Email/password login form
/workspace/logout	POST	Clear workspace session
/workspace/register	GET/POST	Create org + first admin
/workspace/	GET	Dashboard with stats + anomalies
/workspace/workers	GET	Worker table linked to org
/workspace/workers/<id>	GET	Worker detail + community prior
/workspace/identities	GET	Identity table + link form
/workspace/identities/link	POST	Create/update identity (admin/hr)
/workspace/settings	GET/POST	Org info + privacy toggles

5. API Endpoints

All under /api prefix. Protected by API key (X-API-Key header) or Discord OAuth session.

Bot Observer Endpoints

Endpoint	Method	Description
/observer/activity	POST	Log message, voice, or presence activity
/observer/moderation	POST	Log moderation actions (ban, kick, timeout, etc.)
/observer/join-leave	POST	Log member join/leave events
/observer/automod	POST	Log AutoMod trigger events
/observer/role-changes	POST	Log role assignment changes
/observer/ping-join	POST	Log ping-join events
/observer/ml/train	POST	Trigger ML model training pipeline
/observer/ml/anomalies/scan	POST	Run ML anomaly scan
/observer/anomalies/scan	POST	Run rule-based anomaly scan
/observer/ml/burnout-scan	POST	Run burnout risk scan
/observer/ml/federated/train	POST	Train federated model

6. Discord Bot

The Discord bot (bot.py, 82 lines) delegates all logic to bot_core/ package. Uses discord.py library with Intents for full event capture.

Event Handlers

on_message

Capture message content, length, channel, timestamp. Send to /observer/activity.

on_message_edit

Log message edits (future use).

on_voice_state_update

Track voice channel join/leave, compute session duration.

on_member_join

Log member join with guild info. Buffer and send batch to /observer/join-leave.

on_member_remove

Log member leave/possible kick. Buffer and send batch.

on_member_ban

Log ban event with reason and moderator.

on_member_unban

Log unban event.

on_member_update

Detect role changes, log to /observer/role-changes.

Background Tasks

Bot runs 3 background loops: scheduler (scheduled ML training), presence flusher (batch activity every 30s), and join/leave buffer flusher (batch every 30s). Ready handler checks for loops.is_running() before starting.

7. ML & Analytics Layer

The ml/ package provides 7 modules for predictive analytics and scoring.

anomaly.py

Isolation Forest on 28-dim feature vectors (24 hourly bins + 4 message stats). Threshold: -0.15.

burnout.py

Isolation Forest on 7-dim staff feature vector. Threshold: 0.0. Flags 1 in 5 staff members.

federated.py

FedAvg implementation over 2 guilds. 11.8k messages, 98% accuracy. Hour sin/cos features, 80/20 train/test split.

corrector.py

Admin correction feedback. Ridge regression with LOOCV. 4 features, R-squared: 0.71 (was 0.0 with DecisionTree).

features.py

Feature engineering. Functions: user_hourly_profile, user_message_stats, staff_feature_vectors, community_prior_for_worker (5 signals bridging Discord to workspace).

growth.py

Growth prediction module (ML forecasting for join/leave trends).

engine.py

Central ML engine that orchestrates training, tracks last train times, and routes retrain requests.

Community Prior Signals (ml/features.py)

- activity_consistency (0-1): Inverted coefficient of variation of daily message count
- off_hours_ratio (0-1): Proportion of messages outside 09:00-17:00
- anomaly_rate (0-1): Anomalies per 30 days, capped at 10
- score_trajectory (0/0.5/1): Second-half score sum vs first-half
- recent_activity_ratio (0-2): (msgs/day last 7d) / (msgs/day last 30d)

8. Work Engine

The `work_engine/` package connects external project management systems to SkillSync scoring.

Connectors

Jira Connector (`connector_jira.py`)

Syncs tasks from Jira using REST API. Configurable via `JIRA_URL`, `JIRA_EMAIL`, `JIRA_API_TOKEN`, `JIRA_PROJECT` environment variables. Polls every 5 minutes. Creates Task records linked to workers by Jira Account ID via WorkerIdentity bridge.

API Routes (`routes/work.py`)

Provides endpoints for external task ingestion: `POST /api/work/task`, `POST /api/work/sync`, `GET /api/work/tasks`, `GET /api/work/stats`. Supports `external_id` deduplication and source tracking.

9. Dual Dashboard System

SkillSync provides two separate dashboard systems sharing one database, each with independent authentication.

Community Dashboard

Accessed at /. Uses Discord OAuth2 for authentication. Requires DISCORD_CLIENT_ID, DISCORD_CLIENT_SECRET, DISCORD_REDIRECT_URI in .env. Shows: worker stats grid, guild selector sidebar, ML status badges with retrain buttons, federated learning convergence chart, growth analytics (7-day join/leave metrics + hourly chart), anomaly flagged-users table, moderation action log, voice activity summary, hourly activity heatmap.

Workspace Dashboard

Accessed at /workspace/*. Uses email/password authentication via werkzeug.security. Session keys namespaced as ws_* to avoid collision with Discord OAuth. Roles: admin, hr, member. Shows: worker table with identity links, worker detail with community prior card (5 behavioral signals), identity management with inline link form, organisation settings with privacy toggles, work engine anomaly tracking (filtered by source=work_engine).

WorkerIdentity Bridge

The WorkerIdentity model (database.py) links Discord IDs to organizational employee records. Supports Jira Account ID mapping for cross-system scoring. Consent flags (consent_community_prior, consent_federated) give workers control over data sharing. Unique constraints on (org_id, discord_id) and (org_id, org_employee_id) prevent duplicates.

10. Deployment & Infrastructure

Current Setup

Component	Details
Web Server	Flask development server on localhost:5000
Database	PostgreSQL on localhost, 22 tables, migrations via Alembic
Discord Bot	python.exe bot.py, persistent background process
Tunnel	ngrok (pyngrok) at appendage-uptake-aflutter.ngrok-free.dev
Session	Flask sessions via signed cookies, HttpOnly + SameSite=Lax

Environment Variables

Variable	Description
SECRET_KEY	Flask session signing key (required)
DATABASE_URL	PostgreSQL connection string
DISCORD_TOKEN	Discord bot token
DISCORD_CLIENT_ID	Discord OAuth2 client ID
DISCORD_CLIENT_SECRET	Discord OAuth2 client secret
SKILLSYNC_API	Internal API base URL
JIRA_URL	Jira instance URL (optional)
JIRA_EMAIL	Jira API user email
JIRA_API_TOKEN	Jira API token
JIRA_PROJECT	Jira project key

Running the Project

1. Clone repo and copy .env.example to .env, fill in all values
2. Install dependencies: `pip install -r requirements.txt`
3. Ensure PostgreSQL is running with database skillsync
4. Start Flask: `flask run --host=0.0.0.0 --port=5000` (auto-migrates on startup)
5. Start bot: `python bot.py`
6. (Optional) Start ngrok: `pyngrok` or `ngrok http 5000`
7. Access community dashboard at <http://localhost:5000/>
8. Register org at `/workspace/register`, then login at `/workspace/login`